

Roteiro de Apresentacao - Sistema de Mensagens Offline (PPD)

Guia para apresentar o projeto ao professor, focado na **comunicacao** (gRPC + MQTT): a sequencia de arquivos e metodos a mostrar, o que falar em cada um, perguntas provaveis com respostas e um resumo final em topicos.

Ideia central (fixe e repita)

gRPC (RPC) e o canal entre **cliente servidor** (requisito 4). **MQTT/Mosquitto (MOM)** e a **fila de mensagens offline, uma por cliente** (requisito 5). **O cliente NUNCA fala com o MQTT** - so o servidor fala. Essa separacao e o coracao do projeto.

Apresente seguindo **o caminho de uma mensagem** - e a forma mais clara de mostrar a comunicacao.

Parte 0 - Abertura (1 min)

O que falar: "O sistema e um *messenger* com controle de mensagens offline. Sao duas tecnologias da disciplina combinadas: **gRPC** para o transporte remoto cliente-servidor e um **MOM (broker MQTT Mosquitto)** para as filas offline. E um unico executavel Flutter que roda como **Servidor** ou como **Cliente** (seletor de papel na abertura)."

O que mostrar: o diagrama do `README.md` (secao *Arquitetura*):

```
[Cliente ana ]--gRPC--+
                    +--> [Servidor]--MQTT--> [Broker Mosquitto]
[Cliente bruno]--gRPC--+  gRPC + presenca      filas offline (retidas)
```

Parte A - O contrato da comunicacao: `proto/messaging.proto`

Arquivo: `proto/messaging.proto`

O que falar: "Toda a comunicacao gRPC comeca por um contrato `.proto`. Aqui defino o servico `MessageService` com 5 chamadas remotas, **cada operacao no seu proprio metodo**. A partir desse arquivo o `protoc` gera o codigo Dart do cliente e do servidor."

Mostre os 5 RPCs e o tipo de cada um:

- `Register` -> **unary** (req 7: cliente pede para criar sua fila).
- `SendMessage` -> **unary** (req 3 e 6: envia uma mensagem).

- `ReceiveMessages` -> **server-streaming** de `IncomingMessage` (req 3: o servidor *empurra* mensagens).
- `WatchReceipts` -> **server-streaming** de `DeliveryReceipt` (recibos das minhas mensagens).
- `WatchPresence` -> **server-streaming** de `PresenceEvent` (status on/off dos contatos).

Destaque (ponto de design): cada metodo tem **tipos proprios** e **cada stream carrega um unico tipo concreto**. Nao existe uma mensagem "coringa" com `oneof` nem `switch` para descobrir o que chegou - quem roteia cada operacao e o **proprio gRPC pelo nome do metodo**. (Veja a pergunta sobre isso na secao de perguntas: e exatamente o ponto que diferencia este projeto do `dara-game`.)

Parte B - Subindo os dois canais (Servidor):

`lib/state/server_controller.dart`

Arquivo: `lib/state/server_controller.dart` -> metodo `start()`

O que falar: "No papel Servidor faco duas coisas: (1) conecto ao broker MQTT e (2) subo o servidor gRPC."

Mostre as 2 linhas-chave dentro de `start()` :

- `await mqtt.connect( ... )` -> conexao com o MOM.
- `grpc.Server.create(services: [MessageServer(mqtt)])` + `server.serve(port: grpcPort)` -> sobe o servidor gRPC na porta 50051.

Diga: "Repare que **injeta o `MqttService` dentro do `MessageServer`** - e assim que o servidor gRPC usa o MOM."

Parte C - O cliente se conecta via RPC: `lib/rpc/chat_client.dart`

Arquivo: `lib/rpc/chat_client.dart`

O que falar: "Do lado do cliente, encapsulei o stub gerado num wrapper. No construtor crio o `ClientChannel` apontando para o host/porta do servidor."

Mostre:

- `ClientChannel(host, port, ChannelCredentials.insecure())` e `MessageServiceClient(_channel)` .
- Os 4 metodos finos: `register` , `send` , `subscribe` , `watchPresence` .

Frase para fechar: "Esse arquivo e a **unica porta** do cliente para o mundo - ele so conhece gRPC."

Parte D - O coracao: roteamento online/offline em

`lib/server/message_server.dart`

Arquivo: `lib/server/message_server.dart` - **passe mais tempo aqui.**

Mostre **4 metodos nesta ordem**, contando a historia:

1. `register()` (req 7) - "Ao entrar, o cliente chama `Register`. O servidor registra o nome e prepara a fila dele."
2. `sendMessage()` (req 3 e 6) - **o ponto mais importante.** Mostre o `if`:

```
final sink = _online[destino];
if (sink != null) {
  sink.add(Incoming(message: request, fromQueue: false)); // ONLINE → instantaneo
  (req 3)
  return SendReply(ok: true, queued: false);
}
// OFFLINE → vai para a fila do MOM (req 6)
_mqtt.publish(Topics.inbox(destino, id), ChatCodec.encode(request), retain: true);
return SendReply(ok: true, queued: true, queuedId: id);
```

Fale: "Se o destinatario tem um stream aberto (`_online`), entrego **na hora** pelo stream gRPC dele. Se nao tem, **publico uma mensagem retida no broker** - essa e a fila."

3. `receiveMessages()` (req 3) - "Quando o cliente fica online, abre esse stream. Registro o sink em `_online`, marco presenca e **assino o curinga da fila dele no MQTT** - isso faz o broker reentregar tudo que estava acumulado."

```
_online[nome] = controller;
_mqtt.subscribe(Topics.inboxWildcard(nome)); // dispara o "drain" da fila
```

4. `_aoReceberDoMom()` - "Cada mensagem retida que o broker reentrega cai aqui. Repasso ao destinatario marcando `fromQueue: true`, **apago a retida** (`deleteRetained`) e **aviso o remetente com um recibo** (`_notificarEntrega`)."

Conecte com o recibo: `_notificarEntrega()` empurra o `DeliveryReceipt` pelo stream `WatchReceipts` do remetente (ou guarda em `_recibosPendentes` se ele nao tiver esse stream aberto).

Parte E - O MOM: a fila por cliente

Arquivos: `lib/core/mqtt_service.dart` e `lib/core/topics.dart`

O que falar: "A fila não é uma estrutura minha - é o **broker** que guarda. Cada cliente tem o subarvore de tópicos `mom/inbox/<nome>/<id>`."

Mostre em `topics.dart` : `inbox()` , `inboxWildcard()` , `recipientOf()` / `idOf()` .

Mostre em `mqtt_service.dart` :

- `publish( ... , retain: true)` -> enfileirar (a mensagem fica retida no broker).
- `subscribe()` -> ao assinar, o broker reentrega as retidas = esvaziar a fila.
- `deleteRetained()` -> publica payload vazio retido para **remover o item entregue** da fila.
- o stream `mensagens` -> de onde o `MessageServer` escuta.

Destaque: "Uso **JSON** como payload no MQTT de propósito - dá para inspecionar a fila ao vivo com `mosquitto_sub -t 'mom/inbox/#' -v`."

Parte F - O cliente consumindo o stream: `lib/state/chat_controller.dart`

Arquivo: `lib/state/chat_controller.dart`

O que falar e mostrar:

- `init()` -> chama `register` (req 7), abre `watchReceipts` e `watchPresence` e fica online.
- `setOnline(bool)` (req 2) -> **ligar = abrir o stream** `ReceiveMessages` ; **desligar = cancelar o stream**.
- **Dois listeners dedicados, sem** `switch` : `_aoReceberMensagem` escuta `ReceiveMessages` (mostra no chat); `_aoReceberRecibo` escuta `WatchReceipts` (marca a bolha como "entregue"). Cada stream carrega um único tipo, então cada um tem seu próprio `.listen()` - nada de inspecionar tipo.

Frase: "Desligar o botão = fechar meu stream. Sem stream, o servidor cai no `else` do `sendMessage` e enfileira no MOM. É a mesma decisão de um único `if`."

Parte G - Demonstração ao vivo (por último)

1. Suba o broker (terminal bash): `"/c/Program Files/mosquitto/mosquitto.exe" -c broker/mosquitto.conf -v`
2. Abra 1 instância como **Servidor** (mostre o log: "Broker conectado", "gRPC ouvindo em :50051").
3. Abra 2 como **Cliente** (`ana` , `bruno`), adicione contatos.

4. **Online:** ana->bruno chega na hora; bolha "entregue".
 5. **Offline:** desligue bruno, ana envia -> "na fila (offline)". **Mostre no terminal do Mosquitto** a mensagem retida aparecendo (`mosquitto_sub -t "mom/inbox/#" -v`).
 6. **Religue bruno:** mensagens chegam como "recebida da fila" e **as bolhas da ana viram "entregue"**.
-

Perguntas provaveis do professor (e como responder)

1. "Por que usar gRPC E MQTT? Nao dava pra fazer so com um?"

"Sao papeis diferentes que o enunciado pede. O req 4 diz que o cliente acessa o servidor offline via **RPC** -> gRPC. O req 5 pede uma **fila por cliente gerenciada por um MOM** -> MQTT. O gRPC e o transporte; o MQTT e o armazenamento duravel da fila. O cliente so fala gRPC, so o servidor fala MQTT."

2. "Onde fica a fila? E na memoria do servidor?"

"Nao - fica **no broker**, como mensagens retidas em `mom/inbox/<nome>/<id>` . Posso provar com `mosquitto_sub -t 'mom/inbox/#' -v` . Com `persistence true` no `mosquitto.conf` , a fila sobrevive ate a um reinicio do broker." (mostre `Topics.inbox` e `publish(retain:true)`)

3. "Como a mensagem e entregue instantaneamente quando o contato esta online?"

"No `sendMessage` , procuro o stream do destinatario em `_online` . Se existe, faco `sink.add( ... )` direto - vai pelo stream `ReceiveMessages` dele sem passar pela fila." (abra o `if` do `sendMessage`)

4. "Como o servidor sabe que o cliente ficou online para esvaziar a fila?"

"Ficar online = abrir o stream `ReceiveMessages` . No `receiveMessages()` assino `mom/inbox/<nome>/+` no broker, e o MQTT reentrega automaticamente as retidas. Cada uma cai no `_aoReceberDoMom` , que repassa ao cliente e apaga a retida."

5. "Esse stream e bidirecional, como no jogo dara?"

"Nao. Aqui o `ReceiveMessages` e **server-streaming** (servidor -> cliente). O envio e uma chamada **unary** separada, `SendMessage` . Separa 'eu envio' de 'eu recebo'."

5b. "Por que NAO juntou tudo numa mensagem so com `oneof` e um `switch` , como no dara-game?"

"Justamente para não repetir aquele padrão. Aqui **cada operação é um método RPC próprio** (`Register` , `SendMessage` , `ReceiveMessages` , `WatchReceipts` , `WatchPresence`) e **cada stream carrega um único tipo concreto**. Quem decide 'isso é envio / isso é recibo / isso é presença' é o **gRPC pelo nome do método** - não tem mensagem coringa nem `switch` de roteamento. Mensagem e recibo, por exemplo, vêm em streams separados (`ReceiveMessages` -> `IncomingMessage` , `WatchReceipts` -> `DeliveryReceipt`), cada um tratado por um `.listen()` dedicado." (mostre o `proto` e os dois listeners no `chat_controller`)

6. "Como você cria a fila quando o cliente entra (req 7)?"

"Via RPC `Register` . A 'fila' é o subárvore de tópicos do cliente no broker; o `Register` registra o cliente no servidor e o deixa pronto para receber. O que materializa itens na fila é o `publish` retido quando alguém envia para ele offline."

7. "Por que mensagem retida e não uma fila 'de verdade' (ex.: sessão persistente MQTT)?"

"Trade-off consciente: a retida é simples, confiável, **guarda várias mensagens**, e **inspecionável** no terminal e persiste no disco do broker. Sessão persistente exigiria gerenciar conexão/desconexão por cliente e seria mais frágil para a demo."

8. "E se chegarem várias mensagens para o mesmo contato offline?"

"Cada uma recebe um `id` único (`_proximoId++`), virando um tópico distinto `mom/inbox/<nome>/<id>` - sem sobrescrita. No cliente, ordeno por `timestamp` ."

9. "Como o remetente sabe que a mensagem offline foi finalmente entregue?"

"Implementei **recibo de entrega**. Quando o servidor esvazia a fila do destinatário, manda um `DeliveryReceipt` ao remetente pelo stream dedicado `WatchReceipts` , casando pelo `queued_id` . A bolha muda de 'na fila (offline)' para 'entregue'. Se o remetente não tiver o stream aberto, o recibo fica guardado e é entregue quando ele reabre." (mostre `_notificarEntrega` e `_aoReceberRecibo`)

10. "Qual a serialização? Protobuf ou JSON?"

"No gRPC, **Protocol Buffers** (gerado do `.proto`). No MQTT, serializo em **JSON** (`ChatCodec`) - escolha proposital para a fila ficar legível na inspeção."

11. "E múltiplos clientes ao mesmo tempo? Concorrência?"

"O servidor mantém um `Map<nome, StreamController> ( _online )`. Cada `ReceiveMessages` é um stream independente; o roteamento por nome isola os clientes. Dart é single-threaded com loop de eventos, então não há condição de corrida de memória."

12. "Se o servidor cair, o que acontece?"

"As filas (mensagens retidas) persistem no broker. O que é em memória - presença e recibos pendentes - se perde, o que é aceitável para o escopo. As mensagens não."

13. "QoS do MQTT?"

"Uso `atLeastOnce` (QoS 1), garantindo entrega ao menos uma vez para a fila." (no `mqtt_service.publish`)

Roteiro RESUMIDO (cola para apresentar)

1. **Abertura** - messenger offline; gRPC = transporte clienteservidor; MQTT = fila offline por cliente; cliente so fala gRPC.
2. `proto/messaging.proto` - contrato: **5 metodos, 1 por operacao**; `Register` / `SendMessage` (unary), `ReceiveMessages` / `WatchReceipts` / `WatchPresence` (server-stream, cada um com 1 tipo). **Sem oneof, sem switch** - o gRPC roteia pelo nome do metodo (diferenca-chave vs. o `dara-game`).
3. `server_controller.dart` -> `start()` - conecta no MQTT + `Server.create/serve` (50051); injeta `MqttService` no `MessageServer` .
4. `chat_client.dart` - `ClientChannel` insecure + os 5 RPCs; unica porta do cliente.
5. `message_server.dart` (**CORACAO**):
 - o `register()` -> req 7.
 - o `sendMessage()` -> o `if`: online = `sink.add` (req 3); offline = `publish(retain)` (req 6).
 - o `receiveMessages()` -> online abre stream + assina curinga = esvazia fila.
 - o `_aoReceberDoMom()` -> repassa, apaga retida, manda recibo (`_notificarEntrega`).
6. `mqtt_service.dart` + `topics.dart` - a fila e o broker: `mom/inbox/<nome>/<id>` ; `publish(retain)` , `subscribe` , `deleteRetained` ; payload JSON.
7. `chat_controller.dart` - `setOnline` = abrir/fechar `ReceiveMessages` (req 2); dois listeners dedicados (`_aoReceberMensagem` e `_aoReceberRecibo`), sem `switch` .
8. **Demo** - online instantaneo -> offline na fila (mostrar no `mosquitto_sub`) -> religar -> flush + recibo.
9. **Fechamento** - "RPC para falar com o servidor, MOM para guardar a fila; a decisao online/offline e um unico `if` no `sendMessage` ."